## Department of Information Technology

**Faculty of Engineering Sciences and Technology**

**Hamdard University Karachi**


---

## Combined Laboratory Manual - Solution

# Scripting Programs

**Fall 2026 Revised Edition**

**Part I: DFSP-5208: Scripting Programs**

BS Digital Forensics and Cyber Security

**Part II: CCSP-5213: Scripting Program and Cloud Architecture**

BS Cloud Applications Development and Operations


**Instructor:** Iqbal Uddin Khan

Email: Iqbaluddin.khan@hamdard.edu.pk


Academic Year 2026 | Fall 2026

# Contents

1. How to Use This Solution Pack ..... 3
Part I: DFSP-5208 Solutions (Labs 1-16) ..... 4
Lab 1: Introduction to Python Scripting ..... 5
Lab 2: File Handling and Automation in Python ..... 6
Lab 3: Command-Line Tools with Python (argparse) ..... 8
Lab 4: Fundamentals of Bash Scripting ..... 10
Lab 5: Automating System Tasks with Bash (cron) ..... 12
Lab 6: Web Scraping with Python (BeautifulSoup) ..... 14
Lab 7: Node.js Scripting for File System Automation ..... 16
Lab 8: Deploying Node.js Scripts as Serverless Functions ..... 18
Lab 9: REST API Interaction: Fetching and Parsing JSON Data ..... 20
Lab 10: Automating Security Workflows with APIs ..... 23
Lab 11: Error Handling and Debugging in Scripts ..... 25
Lab 12: Logging and Monitoring in Security Scripts ..... 27
Lab 13: Introduction to Docker: Containerizing Forensic Tools ..... 29
Lab 14: Automating Container Deployment ..... 31
Lab 15: CI/CD Pipeline Automation ..... 32
Lab 16: Capstone Lab: Complete Forensic Pipeline ..... 34
Part II: CCSP-5213 Solutions (Labs 1-16) ..... 37
Lab 16 (CCSP): Complete Cloud-Native Pipeline ..... 37
2. Marking Notes ..... 40

# 1. How to Use This Solution Pack

This file gives the instructor's solution for **all 32 labs** (Part I DFSP-5208, Part II CCSP-5213) of the Combined Laboratory Manual. It is written for use with **Lab_Manual_Verification_Report_Fall_2026.docx**, which lists 35 defects found in the manual. Where the manual's own code fails or gives a wrong result, the solution uses **corrected code**, and says so.

- **Tested code:** most scripts below were run in a sandbox (Python 3.12.3, Node 22, Bash 5.2) and the printed output is shown. Output with timestamps will differ on your machine.
- **Not run here:** anything needing Docker, faasd, Jenkins, GitHub Actions, public websites or extra npm packages. These are marked **Reasoned** and should be tried once on a lab PC before term.
- **Checkpoints:** each lab ends with what a marker should look for. Marks are not allocated here; use the rubrics in the Assessment Pack.
- **Personalisation:** ask students to include a personal parameter (for example threshold = last two digits of the roll number) and a screenshot with the system clock.
- **CCSP labs** that match a DFSP lab say "same as DFSP Lab N" and list only the differences.

| Status label | Meaning |
|---|---|
| Executed | Code run in the sandbox and output observed |
| Reasoned | Based on documented behaviour; could not be run here |
| Mixed | Part executed, part reasoned |

| Folder layout used in the solutions | Purpose |
|---|---|
| `~/forensic-scripting/labNN` (DFSP) or `~/cloud-scripting/labNN` (CCSP) | Same as the manual |
| `.venv/` inside lab01 | Re-used by later labs; never committed |

# Part I: DFSP-5208 Solutions (Labs 1-16)

# Lab 1: Introduction to Python Scripting

**Recommended lecture week:** Week 1

> **Verification status: Executed (script); Reasoned (installation)**
> On Ubuntu 24.04 `apt install python3` gives Python 3.12, not 3.13. Accept 3.12 or newer for the screenshot.

## Solution

The script from the manual is correct. Add a hash of the script to practise the integrity idea used all semester (optional extension):

```
import hashlib, pathlib, platform, datetime
print("Scripting environment ready.")
print(f"Python {platform.python_version()} running on {platform.system()}")
print("Course: DFSP-5208 - Scripting Programs")
print("UTC:",
datetime.datetime.now(datetime.timezone.utc).isoformat(timespec="seconds"))
print("SHA256 of this script:",
hashlib.sha256(pathlib.Path(__file__).read_bytes()).hexdigest())
```

**Output observed when the corrected code was run:**

```
Scripting environment ready.
Python 3.12.3 running on Linux
Course: DFSP-5208 – Scripting Programs
```

## Written answers

**Reflection (model answer).** A Python script is read and executed line by line by the interpreter at run time (internally it is compiled to bytecode), so it can be edited and re-run at once. A C or Java program is compiled before it runs, which gives faster execution and type checking but adds a build step. For short forensic tasks such as parsing a log, the quick edit-and-run cycle of a script is worth more than raw speed.

**`.gitignore` (model answer):**

```
.venv/
__pycache__/
.env
*.log
evidence/
```

## Checkpoints

- `python3 --version` shows 3.12 or newer, screenshot has a clock.
- `.venv` exists and is listed in `.gitignore`; `git status` does not show it.
- Reflection mentions interpretation versus compilation and gives a forensic reason.

**CCSP Lab 1:** same as above; the course line reads `CCSP-5213 - Scripting Program and Cloud Architecture`, and the folder is `~/cloud-scripting/lab01`.

# Lab 2: File Handling and Automation in Python

**Recommended lecture week:** Week 2

> **Verification status: Executed**
> The manual's code runs. The solution adds a fixed random seed (so every student can check counts), UTC timestamps, UTF-8 and the WARN counter required by the task.

## generate_sample.py

```
import csv, random, datetime

random.seed(42)                       # same data on every run, so answers can be
checked
now = datetime.datetime.now(datetime.timezone.utc)
rows = []
for i in range(50):
    ts = now - datetime.timedelta(minutes=i * 7)
    rows.append([ts.isoformat(timespec="seconds"), f"host-{random.randint(1, 5)}",
                 random.choice(["INFO", "WARN", "ERROR"]), f"event id {i}"])
with open("sample_data.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["timestamp", "host", "level", "message"])
    writer.writerows(rows)
print("sample_data.csv generated")
```

## filter_errors.py (with WARN count and UTC run history)

```
import csv, datetime
from pathlib import Path

src, dst = Path("sample_data.csv"), Path("errors_only.csv")
counts = {"INFO": 0, "WARN": 0, "ERROR": 0}
with src.open(newline="", encoding="utf-8") as inp, dst.open("w", newline="",
encoding="utf-8") as out:
    reader = csv.DictReader(inp)
    writer = csv.DictWriter(out, fieldnames=reader.fieldnames)
    writer.writeheader()
    for row in reader:
        counts[row["level"]] = counts.get(row["level"], 0) + 1
        if row["level"] == "ERROR":
            writer.writerow(row)
print(f"{counts['ERROR']} ERROR rows written to {dst}; {counts['WARN']} WARN rows
seen; {counts['INFO']} INFO rows seen")

with open("run_history.log", "a", encoding="utf-8") as log:

log.write(f"{datetime.datetime.now(datetime.timezone.utc).isoformat(timespec='seconds'
)} - filter_errors.py executed\n")
```

**Output observed when the corrected code was run:**

```
sample_data.csv generated
12 ERROR rows written to errors_only.csv; 16 WARN rows seen; 22 INFO rows seen
timestamp,host,level,message
2026-10-09T15:41:36+00:00,host-1,ERROR,event id 3
2026-10-09T15:06:36+00:00,host-2,ERROR,event id 8
2026-10-09T10:53:34+00:00 - filter_errors.py executed
2026-10-09T10:53:34+00:00 - filter_errors.py executed
```

### Written answer

`with open(...)` closes the file automatically, even if an exception occurs in the block. Calling `open()` and `close()` by hand risks leaking file handles and leaving files half-written when the code between them fails, which matters when handling evidence files.

### Checkpoints

- `errors_only.csv` has a header and only ERROR rows; counts add up to 50 (12 + 16 + 22 with seed 42).
- `run_history.log` grows by one line per run (append mode).
- WARN count is printed in the summary line.

**CCSP Lab 2:** same code. The data is described as cloud usage events; accept it, but for Assignment 1 consistency also let students create `resources.csv` (`id,region,status`) with the same loop.

# Lab 3: Command-Line Tools with Python (argparse)

**Recommended lecture week:** Week 3

> **Verification status: Executed**
> The manual's tool reports "0 rows written" while writing 23 rows. `writer.writerow()` returns the number of characters written, so `... is None` is never true. The corrected tool counts in a loop.

## log_filter_cli.py (corrected, with --limit and logging)

```
#!/usr/bin/env python3
"""Filter a CSV of log records by level. Usage: log_filter_cli.py --input IN --output
OUT [--level L] [--limit N]"""
import argparse, csv, logging, sys
from pathlib import Path

logging.basicConfig(level=logging.INFO,
format="%(asctime)s %(levelname)s %(message)s")


def main():
    parser = argparse.ArgumentParser(description="Filter CSV records by log level.")
    parser.add_argument("--input", required=True, help="Path to source CSV")
    parser.add_argument("--output", required=True, help="Path to write filtered CSV")
    parser.add_argument("--level", default="ERROR", choices=["INFO", "WARN", "ERROR"],
help="Level to keep")
    parser.add_argument("--limit", type=int, default=0, help="Stop after N matching
rows (0 = no limit)")
    args = parser.parse_args()

    src, dst = Path(args.input), Path(args.output)
    if not src.exists():
        logging.error("Input file not found: %s", src)
        return 1
    dst.parent.mkdir(parents=True, exist_ok=True)

    written = skipped = 0
    with src.open(newline="", encoding="utf-8") as inp, dst.open("w", newline="",
encoding="utf-8") as out:
        reader = csv.DictReader(inp)
        writer = csv.DictWriter(out, fieldnames=reader.fieldnames)
        writer.writeheader()
        for line_no, row in enumerate(reader, start=2):       # line 1 is the header
            level = row.get("level")
            if level is None:
                skipped += 1
                logging.warning("line %d skipped: no 'level' column value", line_no)
                continue
            if level == args.level:
                writer.writerow(row)
                written += 1
                if args.limit and written >= args.limit:
                    break
    print(f"{written} rows with level={args.level} written to {dst} ({skipped}
skipped)")
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

**Output observed when the corrected code was run:**

```
$ python3 log_filter_cli.py --input sample_data.csv --output out/warn.csv --level WARN
--limit 3
3 rows with level=WARN written to out/warn.csv (0 skipped)

$ python3 log_filter_cli.py --input nope.csv --output x.csv; echo $?
2026-10-09 10:53:34,959 ERROR Input file not found: nope.csv
1
```

### --help output

```
usage: l03_log_filter_cli.py [-h] --input INPUT --output OUTPUT
                             [--level {INFO,WARN,ERROR}] [--limit LIMIT]

Filter CSV records by log level.

options:
  -h, --help            show this help message and exit
  --input INPUT         Path to source CSV
  --output OUTPUT       Path to write filtered CSV
  --level {INFO,WARN,ERROR}
                        Level to keep
  --limit LIMIT         Stop after N matching rows (0 = no limit)
```

### README.md for the extra flag

```
# log_filter_cli
Filters a CSV of log records by level.

    python3 log_filter_cli.py --input IN.csv --output OUT.csv [--level
INFO|WARN|ERROR] [--limit N]

--limit N   stop after N matching rows (default 0 = no limit)
```

### Checkpoints

- Printed count equals `wc -l` of the output minus one (the earlier version printed 0).
- Missing input gives a message and exit code 1.
- `--help` lists all four options; a Git commit exists for the lab.

**CCSP Lab 3:** same script; the description text says "cloud CSV records".

# Lab 4: Fundamentals of Bash Scripting

**Recommended lecture week:** Week 4

> **Verification status: Executed**
> The manual's script moves itself into `archive_*/other/` when run in the same folder, miscounts items and overwrites silently on a name clash. The corrected script fixes all three.

## organize.sh (corrected)

```
#!/usr/bin/env bash
# organize.sh: move files in a folder into dated archive sub-folders by extension.
# set -e: exit on any error; set -u: error on unset variables; set -o pipefail: a
pipeline fails if any stage fails.
set -euo pipefail

TARGET_DIR="${1:-.}"
[ -d "$TARGET_DIR" ] || { echo "Not a directory: $TARGET_DIR" >&2; exit 1; }
DATE_TAG=$(date +%Y-%m-%d)
ARCHIVE="${TARGET_DIR}/archive_${DATE_TAG}"
SELF="$(basename "${BASH_SOURCE[0]}")"

mkdir -p "${ARCHIVE}/logs" "${ARCHIVE}/csv" "${ARCHIVE}/other"
n_logs=0; n_csv=0; n_other=0

move_safe() {            # never overwrite: on a name clash, prefix the time so both
copies are kept
  local src="$1" dest_dir="$2" base target
  base="$(basename "$src")"; target="${dest_dir}/${base}"
  [ -e "$target" ] && target="${dest_dir}/$(date +%H%M%S)_${base}"
  mv -- "$src" "$target"
}

for f in "${TARGET_DIR}"/*; do
  [ -f "$f" ] || continue
  [ "$(basename "$f")" = "$SELF" ] && continue          # never move this script
itself
  case "$f" in
    *.log) move_safe "$f" "${ARCHIVE}/logs";   n_logs=$((n_logs + 1)) ;;
    *.csv) move_safe "$f" "${ARCHIVE}/csv";     n_csv=$((n_csv + 1)) ;;
    *)     move_safe "$f" "${ARCHIVE}/other"; n_other=$((n_other + 1)) ;;
  esac
done

echo "Organized into ${ARCHIVE}: logs=${n_logs} csv=${n_csv} other=${n_other}"
```

**Output observed when the corrected code was run:**

```
Organized into ./archive_2026-10-09: logs=2 csv=1 other=1
(second run with a name clash, both copies kept:)
Organized into ./archive_2026-10-09: logs=1 csv=0 other=0
105358_a.log   a.log   my file.log
```

## Written answer

Without `set -euo pipefail` the script keeps running after a failure. For example, if `mv` fails, the final `echo` still reports success; an unset variable such as `TARGET_DIR` becomes an empty string and a command like `rm -rf "$TARGET_DIR"/*` can then act on the wrong folder; and in `a | b` only the last command's status is checked, so a failed first stage is hidden.

### Checkpoints

- Script is not moved; per-category counts are correct; a file with a space in its name is moved.
- Student explains `-e`, `-u`, `pipefail` in comments.

**CCSP Lab 4:** identical.

# Lab 5: Automating System Tasks with Bash (cron)

**Recommended lecture week:** Week 5

> **Verification status: Executed (script); Reasoned (cron timing)**
> The manual backs up the whole project folder including every `.venv`, has no checksum and calls a path that does not exist. The corrected script excludes `.venv`, records SHA-256, verifies the archive and applies 7-day retention.

## backup.sh

```
#!/usr/bin/env bash
# backup.sh: timestamped, verified backup with 7-day retention. Paths can be
overridden for testing.
set -euo pipefail
SRC_DIR="${SRC_DIR:-$HOME/forensic-scripting}"       # Cloud track: $HOME/cloud-
scripting
BACKUP_ROOT="${BACKUP_ROOT:-$HOME/forensic-backups}" # Cloud track: $HOME/cloud-
backups
TS=$(date -u +%Y%m%d_%H%M%S)
ARCHIVE="${BACKUP_ROOT}/backup_${TS}.tar.gz"
mkdir -p "$BACKUP_ROOT"

tar --exclude='.venv' --exclude='node_modules' -czf "$ARCHIVE" -C "$SRC_DIR" .
( cd "$BACKUP_ROOT" && sha256sum "$(basename "$ARCHIVE")" >> SHA256SUMS )
tar -tzf "$ARCHIVE" > /dev/null                        # verify the archive can be read
echo "$(date -u +%FT%TZ) backup_${TS}.tar.gz created and verified" >>
"$BACKUP_ROOT/backup_history.log"

find "$BACKUP_ROOT" -name 'backup_*.tar.gz' -mtime +7 -delete   # retention: 7 days
```

**Output observed when the corrected code was run:**

```
SHA256SUMS  backup_20261009_105335.tar.gz  backup_history.log
$ sha256sum -c SHA256SUMS
backup_20261009_105335.tar.gz: OK
$ tar -tzf backup_*.tar.gz        # .venv content absent
./
./f.txt
(a file older than 7 days was removed by the find line)
```

## Crontab

```
# demonstration: every 10 minutes
*/10 * * * * $HOME/forensic-scripting/lab05/backup.sh >> $HOME/forensic-
backups/cron.out 2>&1
# real use: nightly at 02:00
0 2 * * * $HOME/forensic-scripting/lab05/backup.sh >> $HOME/forensic-backups/cron.out
2>&1
```

On WSL without systemd, `sudo service cron start` is needed after each restart; Windows students without WSL may use Task Scheduler (`schtasks`). Two unattended runs should appear in `backup_history.log` after 20 minutes.

## Checkpoints

- History log shows at least two runs at 10-minute spacing; `sha256sum -c` says OK.
- Retention line uses `-mtime +7 -delete`; student explains why the log redirect (`>> cron.out 2>&1`) matters (cron has no terminal).

**Cloud track (CCSP-5213, BS Cloud Application Development & Operations)**

**CCSP Lab 5:** same script with `SRC_DIR=$HOME/cloud-scripting` and `BACKUP_ROOT=$HOME/cloud-backups`. Add the cloud CLI step required by the outline, against an emulator (Moto server on port 5000): `aws --endpoint-url http://localhost:5000 s3 mb s3://lab-backups` then `aws --endpoint-url http://localhost:5000 s3 cp "$ARCHIVE" s3://lab-backups/` (dummy credentials `test`/`test`). Reasoned.

# Lab 6: Web Scraping with Python (BeautifulSoup)

**Recommended lecture week:** Week 6

> **Verification status: Executed against a local test page**
> Public sites were not reachable. The parsing and error handling were run against a local page with one good and one malformed item.

### scraper.py (with timeout handling and tolerant parsing)

```
import csv, sys, time
import requests
from bs4 import BeautifulSoup

URL = sys.argv[1] if len(sys.argv) > 1 else "https://books.toscrape.com/"
HEADERS = {"User-Agent": "Lab06-Student-Scraper/1.0 (course exercise)"}

try:
    resp = requests.get(URL, headers=HEADERS, timeout=10)
    resp.raise_for_status()
except requests.exceptions.Timeout:
    sys.exit(f"Timed out fetching {URL}")
except requests.exceptions.RequestException as exc:
    sys.exit(f"Request failed: {exc}")

soup = BeautifulSoup(resp.text, "html.parser")
rows = []
for book in soup.select("article.product_pod"):
    link = book.h3.a if book.h3 else None
    price = book.select_one(".price_color")
    if link is None or price is None:                    # tolerate a missing element
        print("skipped one malformed item", file=sys.stderr)
        continue
    rows.append([link["title"], price.text.strip()])

with open("scraped_books.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["title", "price"])
    writer.writerows(rows)
print(f"Saved {len(rows)} records to scraped_books.csv")
time.sleep(1)                                            # one request, then a
polite pause before any next page
```

**Output observed when the corrected code was run:**

```
$ python3 scraper.py http://127.0.0.1:8765/
skipped one malformed item
Saved 1 records to scraped_books.csv

$ python3 scraper.py http://127.0.0.1:1/ ; echo $?
Request failed: HTTPConnectionPool(host='127.0.0.1', port=1): Max retries exceeded ...
Connection refused
1
```

On the real site (`https://books.toscrape.com/`) the first page returns 20 books.

### Written answer

`robots.txt` is a file at the root of a website that states which paths automated clients may or may not fetch. It is a convention, not a security control, but ignoring it can breach a site's terms and, depending on the country and data, the law. Rate limiting matters because many fast requests can slow or take

down a small server and may be treated as abuse; a clear `User-Agent` and a pause between requests keep the scraper polite and identifiable.

### Checkpoints

- Student shows the `robots.txt` check; `try/except` wraps `requests.get`; a pause sits between requests.
- CSV has a header and one row per book.

**CCSP Lab 6:** same code. If the cloud status page is used, prefer its RSS/JSON feed over HTML when one exists.

# Lab 7: Node.js Scripting for File System Automation

**Recommended lecture week:** Week 7

> **Verification status: Executed**
> The manual's renamer stacks a second timestamp on a second run and exits 0 when the folder is missing. The corrected version skips files already stamped, is a dry run unless `--apply` is given and never overwrites.

```
// Usage: node rename_files.js <dir> [--apply]   (dry run by default; never
overwrites)
const fs = require('fs/promises');
const path = require('path');

const STAMPED = /^\d{4}-\d{2}-\d{2}T\d{2}-\d{2}-\d{2}-\d{3}Z_/;

async function renameFiles(dir, apply) {
  const stamp = new Date().toISOString().replace(/[:.]/g, '-');
  let count = 0, skipped = 0;
  for (const file of await fs.readdir(dir)) {
    const oldPath = path.join(dir, file);
    if (!(await fs.stat(oldPath)).isFile()) continue;
    if (STAMPED.test(file)) { skipped++; continue; }              // already has a
timestamp prefix
    const newPath = path.join(dir, `${stamp}_${file}`);
    try { await fs.access(newPath); console.error(`refusing to overwrite ${newPath}`);
continue; } catch {}
    console.log(`${apply ? 'RENAME' : 'DRY RUN'} ${file} ->
${path.basename(newPath)}`);
    if (apply) await fs.rename(oldPath, newPath);
    count++;
  }
  console.log(`${apply ? 'Renamed' : 'Would rename'} ${count} files, skipped
${skipped}, in ${dir}`);
}

const args = process.argv.slice(2);
renameFiles(args.find(a => !a.startsWith('--')) || '.', args.includes('--apply'))
  .catch(err => { console.error('Error:', err.message); process.exit(1); });
```

**Output observed when the corrected code was run:**

```
$ node rename_files.js sf
DRY RUN a.txt -> 2026-10-09T10-53-35-092Z_a.txt
DRY RUN b.csv -> 2026-10-09T10-53-35-092Z_b.csv
Would rename 2 files, skipped 0, in sf
$ node rename_files.js sf --apply
Renamed 2 files, skipped 0, in sf
$ node rename_files.js sf --apply
Renamed 0 files, skipped 2, in sf
$ node rename_files.js nodir; echo $?
Error: ENOENT: no such file or directory, scandir 'nodir'
1
```

## Comparison table (3 rows)

| Task | Python | Node.js |
|---|---|---|
| List a folder | `os.listdir(p)` / `Path(p).iterdir()` | `await fs.readdir(p)` |
| Rename a file | `os.rename(a, b)` / `Path.rename()` | `await fs.rename(a, b)` |
| Is it a file? | `Path(p).is_file()` | `(await fs.stat(p)).isFile()` |

Main difference: Node's `fs/promises` calls are asynchronous and need `await`; Python's are synchronous by default.

### Checkpoints

- Dry run first, then `--apply`; second `--apply` changes nothing.
- A feature branch and pull request exist for the lab (Git thread added in the revised notes).

**CCSP Lab 7:** identical logic; the script is called `rename_cloud_files.js`.

# Lab 8: Deploying Node.js Scripts as Serverless Functions

**Recommended lecture week:** Week 8

> **Verification status: Executed (handlers via local harness); Reasoned (faasd)**
> faasd CE is licensed for non-commercial, personal use, needs Linux with systemd, and `faas-cli up` needs a registry. Offer the local harness as the graded route and faasd as an optional extension.

## Tier 1 (required): local harness

```
// Local harness that mimics the OpenFaaS node template, so the function can be tested
without faasd.
const file = process.argv[2];
const handler = require(path = require('path').resolve(file));
const body = process.argv[3] ?? '';
const ctx = { status(code) { return { succeed: b => ({ status: code, body:
b }) }; } };
handler({ body }, ctx).then(r => console.log(JSON.stringify(r)));
```

## DFSP handler.js (IOC list extended)

```
'use strict'
const IOCS = ['malicious.exe', '203.0.113.99', 'suspicious-domain.test',
              'evil-update.example', '198.51.100.23', 'cmd.exe /c powershell -enc'];
// two or more added

module.exports = async (event, context) => {
  const text = (event.body || '').toString().toLowerCase();
  const hits = IOCS.filter(ioc => text.includes(ioc.toLowerCase()));
  return context.status(200).succeed({ scanned: true, hits, matched: hits.length >
0 });
}
```

**Output observed when the corrected code was run:**

```
$ node harness.js handler_dfsp.js 'beacon to 198.51.100.23 seen'
{"status":200,"body":{"scanned":true,"hits":["198.51.100.23"],"matched":true}}
$ node harness.js handler_dfsp.js 'all quiet'
{"status":200,"body":{"scanned":true,"hits":[],"matched":false}}
```

## Tier 2 (optional): faasd

- Run `docker run -d -p 5000:5000 --name registry registry:2`.
- In `ioc-alert.yml` set `image: 127.0.0.1:5000/ioc-alert:latest`, then `faas-cli up -f ioc-alert.yml`.
- Invoke with `curl -X POST http://127.0.0.1:8080/function/ioc-alert -d 'connection from 203.0.113.99 detected'`; expect `"matched":true`.
- Check the faasd handbook for the registry and pull-policy settings for your faasd version, and confirm the CE EULA allows use in your lab.

## Checkpoints

- Both a match and a no-match output are shown, with at least two extra IOCs added.
- Student can state what "stateless" means for the function.

> **Cloud track (CCSP-5213, BS Cloud Application Development & Operations)**
> **CCSP Lab 8: usage-alert.** Handler (validates input, threshold from environment), tested with the harness:

```
'use strict'
const THRESHOLD_USD = Number(process.env.THRESHOLD_USD || 75);   // set to the
instructor's value

module.exports = async (event, context) => {
  let payload;
  try { payload = JSON.parse(event.body || '{}'); }
  catch (e) { return context.status(400).succeed({ error: 'body must be JSON' }); }
  const cost = Number(payload.costUsd);
  if (!payload.resource || Number.isNaN(cost)) {
    return context.status(400).succeed({ error: 'resource and numeric costUsd are
required' });
  }
  return context.status(200).succeed({ resource: payload.resource, costUsd: cost,
overBudget: cost > THRESHOLD_USD });
}
```

**Output observed when the corrected code was run:**

```
$ node harness.js handler_ccsp.js '{"resource":"vm-01","costUsd":72.5}'
{"status":200,"body":{"resource":"vm-01","costUsd":72.5,"overBudget":false}}      #
default 75
$ THRESHOLD_USD=50 node harness.js handler_ccsp.js '{"resource":"vm-
01","costUsd":72.5}'
{"status":200,"body":{"resource":"vm-01","costUsd":72.5,"overBudget":true}}
$ node harness.js handler_ccsp.js 'timestamp,host'
{"status":400,"body":{"error":"body must be JSON"}}
```

The original handler (`JSON.parse(event.body)` with no guard) throws on non-JSON input; this matters in Lab 16.

# Lab 9: REST API Interaction: Fetching and Parsing JSON Data

**Recommended lecture week: Week 10**

> **Verification status: Executed against a local mock API**
>
> The manual's version has no key, evidence file, hash or status handling. The solution implements the Week 10 workflow. Point `API_URL` at `https://jsonplaceholder.typicode.com` for the public API (not reachable in the sandbox).

## fetch_cases.py

```
import datetime, hashlib, json, os, sqlite3, sys
import requests

BASE = os.getenv("API_URL", "https://jsonplaceholder.typicode.com")
URL = f"{BASE}/posts"


def utc_stamp():
    return datetime.datetime.now(datetime.timezone.utc).strftime("%Y%m%d_%H%M%S")


def fetch(url):
    resp = requests.get(url, timeout=10)
    if resp.status_code != 200:                                  # non-200: report and
stop, never parse an error body
        raise RuntimeError(f"HTTP {resp.status_code} from {url}: {resp.text[:120]}")
    return resp.json()


def save_evidence(data, url):
    ts = utc_stamp()
    fname = f"evidence_{ts}.json"
    with open(fname, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2)
    with open(fname, "rb") as f:
        digest = hashlib.sha256(f.read()).hexdigest()
    with open("custody.log", "a", encoding="utf-8") as log:
        log.write(f"{ts}\t{url}\t{fname}\tSHA256={digest}\n")
    return fname, digest


def store(data):
    conn = sqlite3.connect("case_notes.db")
    with conn:
        conn.execute("""CREATE TABLE IF NOT EXISTS notes (
            id INTEGER PRIMARY KEY, user_id INTEGER, title TEXT, body TEXT)""")
        for item in data[:20]:
            conn.execute("INSERT OR REPLACE INTO notes VALUES (?,?,?,?)",
                         (item["id"], item["userId"], item["title"], item["body"]))
    conn.close()
    return min(len(data), 20)


if __name__ == "__main__":
    try:
        data = fetch(URL)
        print(save_evidence(data, URL))
        print(f"{store(data)} records stored in case_notes.db")
    except requests.exceptions.RequestException as exc:
        sys.exit(f"Network failure: {exc}")
```

```
    except RuntimeError as exc:
        sys.exit(f"API failure: {exc}")
```

### fetch_cases.js (built-in fetch; axios variant below)

```
const BASE = process.env.API_URL || 'https://jsonplaceholder.typicode.com';

async function main() {
  try {
    const res = await fetch(`${BASE}/posts`, { signal: AbortSignal.timeout(10000) });
    if (!res.ok) throw new Error(`HTTP ${res.status}`);          // non-200 handled
explicitly
    const data = await res.json();
    console.table(data.slice(0, 5).map(p => ({ id: p.id, title: p.title })));
  } catch (err) {
    console.error('API error:', err.message);
    process.exitCode = 1;
  }
}
main();
```

**axios variant (as in the manual) with explicit non-200 handling:**

```
const axios = require('axios');
axios.get(url, { timeout: 10000, validateStatus: () => true })
  .then(res => {
    if (res.status !== 200) throw new Error(`HTTP ${res.status}`);
    console.table(res.data.slice(0, 5).map(p => ({ id: p.id, title: p.title })));
  })
  .catch(err => { console.error('API error:', err.message); process.exitCode = 1; });
```

**Output observed when the corrected code was run:**

```
$ python3 fetch_cases.py
('evidence_20261009_105337.json',
'33180390f9ad178d0b4a4011dac0b67ef2c124756b784576e6fe2b5dbf888f76')
20 records stored in case_notes.db
$ cat custody.log
20261009_105337  http://127.0.0.1:8765/posts   evidence_20261009_105337.json
     SHA256=33180390...888f76
$ API_URL=http://127.0.0.1:8765/bad python3 fetch_cases.py ; echo $?
API failure: HTTP 404 from http://127.0.0.1:8765/bad/posts: {"error":"nope"}
1
$ node fetch_cases.js
┌─────────┬────┬─────────────┐
│ (index) │ id │ title       │ ...
$ API_URL=http://127.0.0.1:1 node fetch_cases.js ; echo $?
API error: fetch failed
1
```

Query the database (install the CLI first with `sudo apt install sqlite3`): `sqlite3 case_notes.db "SELECT id, title FROM notes LIMIT 5;"`.

## Checkpoints

- A timestamped evidence file, a matching SHA-256 in `custody.log` (`sha256sum evidence_*.json` agrees), and 20 rows in SQLite.
- Both scripts handle a non-200 reply without crashing and exit non-zero.

> **Cloud track (CCSP-5213, BS Cloud Application Development & Operations)**
>
> **CCSP Lab 9:** same code with the table `tasks` (`/todos`) if the public API is used; the revised notes prefer an emulator. With Moto (`pip install "moto[server]"`, `moto_server -p 5000`): create a bucket with `boto3` (endpoint `http://localhost:5000`, keys `test`/`test`) and fetch `GET http://localhost:5000/moto-api/data.json` as the cloud-state JSON; save it with the same `save_evidence` function. Reasoned; check the status path on your Moto version.

# Lab 10: Automating Security Workflows with APIs

**Recommended lecture week: Week 11**

> **Verification status: Executed against a local SMTP sink**
> MailHog is unmaintained since 2020; the code works with any SMTP sink on port 1025 (Mailpit or `python -m aiosmtpd -n -l localhost:1025`).

## alert_workflow.py: one consolidated email

```
import smtplib, sys
from email.mime.text import MIMEText

SMTP_HOST, SMTP_PORT = "localhost", 1025
IOCS = ["malicious.exe", "203.0.113.99"]
LOG_LINES = [
    "user admin logged in from 10.0.0.5",
    "outbound connection to 203.0.113.99 flagged",
    "process malicious.exe started by user bob",
    "scheduled backup completed",
    "dns query for example.org",
]


def scan_line(line):
    return [i for i in IOCS if i in line]


def send_summary(findings):
    body = "IOC summary\n" + "\n".join(f"- line {n}: {hits} -> {line}" for n, hits,
line in findings)
    msg = MIMEText(body)
    msg["Subject"] = f"Forensic Alert: {len(findings)} IOC match(es)"
    msg["From"], msg["To"] = "soc-automation@lab.local", "analyst@lab.local"
    with smtplib.SMTP(SMTP_HOST, SMTP_PORT, timeout=10) as server:
        server.send_message(msg)


if __name__ == "__main__":
    findings = [(n, h, l) for n, l in enumerate(LOG_LINES, 1) if (h := scan_line(l))]
    if not findings:
        print("No IOC matches, no alert sent")
        sys.exit(0)
    try:
        send_summary(findings)                                  # ONE consolidated
email, not one per line
    except (OSError, smtplib.SMTPException) as exc:
        sys.exit(f"Alert failed: {exc}")
    print(f"Consolidated alert sent: {len(findings)} matches in {len(LOG_LINES)}
lines")
```

**Output observed when the corrected code was run:**

```
Consolidated alert sent: 2 matches in 5 lines

Message received by the sink:
Subject: Forensic Alert: 2 IOC match(es)
IOC summary
- line 2: ['203.0.113.99'] -> outbound connection to 203.0.113.99 flagged
- line 3: ['malicious.exe'] -> process malicious.exe started by user bob
```

### alert_workflow.js (nodemailer, consolidated)

```
const nodemailer = require('nodemailer');
const IOCS = ['malicious.exe', '203.0.113.99'];
const LINES = ['user admin logged in', 'outbound connection to 203.0.113.99 flagged',
'process malicious.exe started', 'backup done', 'dns query'];

async function main() {
  const findings = LINES.map((l, i) => ({ n: i + 1, l, hits: IOCS.filter(x =>
l.includes(x)) })).filter(f => f.hits.length);
  if (!findings.length) return console.log('No IOC matches, no alert sent');
  const transporter = nodemailer.createTransport({ host: 'localhost', port: 1025,
secure: false });
  await transporter.sendMail({
    from: 'soc-automation@lab.local', to: 'analyst@lab.local',
    subject: `Forensic Alert: ${findings.length} IOC match(es)`,
    text: findings.map(f => `- line ${f.n}: ${f.hits.join(', ')} ->
${f.l}`).join('\n'),
  });
  console.log(`Consolidated alert sent: ${findings.length} matches`);
}
main().catch(e => { console.error('Alert failed:', e.message); process.exit(1); });
```

Reasoned (nodemailer not installable in the sandbox). Run `npm install nodemailer` first.

### Extension for the Week 11 notes (optional but recommended)

```
import sqlite3, datetime
conn = sqlite3.connect("alerts.db")
conn.execute("CREATE TABLE IF NOT EXISTS alerts (entity TEXT, ts TEXT, action TEXT)")
def already_alerted(entity, hours=24):
    cutoff = (datetime.datetime.now(datetime.timezone.utc) -
datetime.timedelta(hours=hours)).isoformat()
    return conn.execute("SELECT 1 FROM alerts WHERE entity=? AND ts>? LIMIT 1",
(entity, cutoff)).fetchone() is not None
# after a successful send: conn.execute("INSERT INTO alerts VALUES (?,?,?)", (entity,
now_iso, "email")); conn.commit()
```

Then schedule `alert_workflow.py` with cron `*/30 * * * *` and show that the second run does not send a second email.

### Checkpoints

- The sink (or MailHog UI) shows **one** email with both matches.
- With the extension: `SELECT * FROM alerts;` shows rows, and the second run prints "already alerted".

> **Cloud track (CCSP-5213, BS Cloud Application Development & Operations)**
>
> **CCSP Lab 10:** same structure with five sample resources, e.g. `[("vm-01", 72.5), ("db-02", 20.0), ("vm-03", 130.0), ("cache-04", 49.99), ("vm-05", 51.0)]` and `BUDGET_USD = 50`. The consolidated email lists vm-01, vm-03 and vm-05 (cost > 50). Subject `Cloud Cost Alert: 3 resources over budget`.

# Lab 11: Error Handling and Debugging in Scripts

**Recommended lecture week: Week 12**

> **Verification status: Executed**
> The manual does not supply `messy_log.csv` and its log has no timestamps; the CCSP retry helper catches the wrong exception class.

## messy_log.csv (3 malformed rows)

```
timestamp,severity
2026-03-04T10:00:00,3
2026-03-04T10:01:00,high
2026-03-04T10:02:00,
2026-03-04T10:03:00,2
2026-03-04T10:04:00,9x
```

## parse script (timestamped log, custom exception, correct retry)

```
import csv, logging, time

logging.basicConfig(filename="parse_errors.log", level=logging.WARNING,
                    format="%(asctime)s %(levelname)s %(message)s")


class RetryExhausted(Exception):
    """Raised when every attempt of a retried operation has failed."""


def parse_row(row):
    try:
        return {"timestamp": row["timestamp"], "severity": int(row["severity"])}
    except (KeyError, ValueError, TypeError) as exc:
        logging.warning("Skipped malformed row %r: %s", row, exc)
        return None


def with_retry(fn, attempts=3, delay=1.0, retry_on=(OSError,)):
    """Retry fn on transient errors. requests' exceptions subclass OSError, NOT the
builtin ConnectionError."""
    for i in range(1, attempts + 1):
        try:
            return fn()
        except retry_on as exc:
            logging.warning("Attempt %d/%d failed: %s", i, attempts, exc)
            time.sleep(delay * 2 ** (i - 1))
    raise RetryExhausted(f"all {attempts} attempts failed")


if __name__ == "__main__":
    results = []
    try:
        with open("messy_log.csv", newline="", encoding="utf-8") as f:
            for row in csv.DictReader(f):
                parsed = parse_row(row)
                if parsed:
                    results.append(parsed)
    except FileNotFoundError:
        logging.error("messy_log.csv not found")
        raise SystemExit("messy_log.csv not found")
    print(f"Successfully parsed {len(results)} rows; see parse_errors.log for skipped
rows")
```

**Output observed when the corrected code was run:**

```
Successfully parsed 2 rows; see parse_errors.log for skipped rows

$ cat parse_errors.log
2026-10-09 10:53:38,467 WARNING Skipped malformed row {'timestamp': '2026-03-
04T10:01:00', 'severity': 'high'}: invalid literal for int() with base 10: 'high'
2026-10-09 10:53:38,467 WARNING Skipped malformed row {'timestamp': '2026-03-
04T10:02:00', 'severity': ''}: ...
2026-10-09 10:53:38,467 WARNING Skipped malformed row {'timestamp': '2026-03-
04T10:04:00', 'severity': '9x'}: ...
```

The fragile version raises `ValueError: invalid literal for int() with base 10: 'high'` on the second row (observed).

## Debugging steps (VS Code and pdb)

1. Set a breakpoint on the `return` line in `parse_row` (click left of the line number), press F5, and open the Variables panel.
2. Step with F10; when `row['severity']` is `'high'` the value is visible before `int()` fails.
3. Same with the terminal: `python3 -m pdb l11_parse_messy.py`, then `b 12`, `c`, `p row`, `n`.
4. Take the screenshot at the pause showing `row` in the Variables panel.

## Node.js equivalent with async handling

```
function parseRow(row) {
  const sev = Number(row.severity);
  if (row.severity === '' || Number.isNaN(sev)) {
    console.warn(`${new Date().toISOString()} WARN skipped row: invalid severity
"${row.severity}"`);
    return null;
  }
  return { timestamp: row.timestamp, severity: sev };
}
```

## Checkpoints

- Three or more malformed rows are skipped and logged with timestamps; the script finishes.
- Student can explain why `except (KeyError, ValueError)` is better than a bare `except`.
- Debugger screenshot shows the Variables panel with `row`.

> **Cloud track (CCSP-5213, BS Cloud Application Development & Operations)**
>
> **CCSP Lab 11:** use `messy_billing.csv` with columns `resource,cost` and `float(row["cost"])`. The retry helper must catch `requests.exceptions.RequestException` or `OSError`, **not** the built-in `ConnectionError` (confirmed: `issubclass(requests.exceptions.ConnectionError, ConnectionError)` is `False`). Use `with_retry(lambda: requests.get(url, timeout=5).raise_for_status(), retry_on=(requests.exceptions.RequestException,))` from the script above.

# Lab 12: Logging and Monitoring in Security Scripts

**Recommended lecture week: Week 13**

> **Verification status: Executed (Python); Reasoned (Winston, lnav)**
> The manual configures one file handler. The solution adds a JSON handler, a console handler for warnings and a tamper-evidence step.

```
import datetime, json, logging, sys
from logging.handlers import RotatingFileHandler


class JsonFormatter(logging.Formatter):
    def format(self, r):
        return json.dumps({"ts": datetime.datetime.fromtimestamp(r.created,
datetime.timezone.utc).isoformat(),
                           "level": r.levelname, "msg": r.getMessage(), "logger":
r.name,
                           "context": getattr(r, "context", {})})


def make_logger(name="pipeline", max_bytes=1_000_000):
    logger = logging.getLogger(name)
    logger.setLevel(logging.DEBUG)
    text = RotatingFileHandler("pipeline.log", maxBytes=max_bytes, backupCount=3,
encoding="utf-8")
    text.setFormatter(logging.Formatter("%(asctime)s | %(levelname)s | %(message)s"))
    jsonl = RotatingFileHandler("pipeline.jsonl", maxBytes=max_bytes, backupCount=3,
encoding="utf-8")
    jsonl.setFormatter(JsonFormatter())
    console = logging.StreamHandler(sys.stderr)
    console.setLevel(logging.WARNING)
    for h in (text, jsonl, console):
        logger.addHandler(h)
    return logger


if __name__ == "__main__":
    log = make_logger()
    log.info("Pipeline started", extra={"context": {"case": "C-001"}})
    log.warning("Suspicious file skipped: corrupt_evidence.dat")
    log.error("Failed to hash file: permission denied")
```

**Output observed when the corrected code was run:**

```
$ cat pipeline.log
2026-10-09 10:53:38,513 | INFO | Pipeline started
2026-10-09 10:53:38,513 | WARNING | Suspicious file skipped: corrupt_evidence.dat
2026-10-09 10:53:38,514 | ERROR | Failed to hash file: permission denied
$ cat pipeline.jsonl
{"ts": "2026-10-09T10:53:38.513713+00:00", "level": "INFO", "msg": "Pipeline started",
"logger": "pipeline", "context": {"case": "C-001"}}
...
(console, WARNING and above:)
Suspicious file skipped: corrupt_evidence.dat
Failed to hash file: permission denied
```

### Winston with built-in rotation (no extra package)

```
const winston = require('winston');
const logger = winston.createLogger({
  level: 'debug',
  format: winston.format.combine(winston.format.timestamp(), winston.format.json()),
```

```
  transports: [
    new winston.transports.File({ filename: 'pipeline.json.log', maxsize: 1_000_000,
maxFiles: 3 }),
    new winston.transports.Console(),
  ],
});
logger.info('Pipeline started');
logger.warn('Suspicious file skipped: corrupt_evidence.dat');
logger.error('Failed to hash file: permission denied');
```

`winston-daily-rotate-file` is also acceptable if a student prefers daily files (`maxSize: '1m'`).

### Tamper evidence (add to the lab)

```
# set maxBytes small (e.g. 2000) so the log rotates, then hash a CLOSED file:
sha256sum pipeline.log.1 >> log_hashes.txt
# later, verify:
sha256sum -c log_hashes.txt        # OK, or FAILED if one character was changed
```

Explain that the live `pipeline.log` changes with every line, so its hash must be taken only after rotation and stored elsewhere.

### lnav filter

`lnav pipeline.log`, then press `e` / `E` to jump to the next/previous error line, or `:filter-in WARNING|ERROR` to show only those lines. Screenshot the filtered view. (Reasoned; keys vary slightly by lnav version.)

### Checkpoints

- Text log, JSON log and a rotated file all exist; one message is at the correct level.
- A hash of a closed log file is recorded and re-verified.

**CCSP Lab 12:** same code; messages become "Deployment pipeline started", "Resource vm-01 approaching budget limit", "Failed to provision resource: quota exceeded". Add `"resource"` and `"region"` to the JSON `context`.

# Lab 13: Introduction to Docker: Containerizing Forensic Tools

**Recommended lecture week: Week 14**

> **Verification status: Reasoned (no Docker in the sandbox); the script inside was executed**
> The manual mounts `-v $(pwd)/out:/app`, which hides the code inside the image. Mount a sub-folder instead.

## Dockerfile (corrected)

```
FROM python:3.12-slim
WORKDIR /app
COPY log_filter_cli.py sample_data.csv ./
RUN useradd --create-home appuser && mkdir -p /app/out && chown appuser /app/out
USER appuser
ENTRYPOINT ["python", "log_filter_cli.py"]
CMD ["--input", "sample_data.csv", "--output", "/app/out/output.csv", "--level",
"ERROR"]
```

Use `python:3.13-slim` if your lab image is 3.13; the version is not important for the exercise. A `requirements.txt` is not needed because the script uses only the standard library. Add `.dockerignore`:

```
.venv/
.git/
__pycache__/
.env
*.log
out/
```

## Build and run

```
docker build -t forensic-log-filter:1.0 .
mkdir -p out
docker run --rm --user "$(id -u):$(id -g)" -v "$PWD/out:/app/out" forensic-log-
filter:1.0
cat out/output.csv | head
# PowerShell: -v "${PWD}/out:/app/out" (and omit --user)
docker images forensic-log-filter
docker history forensic-log-filter:1.0
```

Expected: the console prints `N rows with level=ERROR written to /app/out/output.csv (0 skipped)` and `out/output.csv` appears on the host. To compare with the host run, create the host file with the same arguments and check `sha256sum out/output.csv host_output.csv`; the two hashes must match.

## Why the original run failed

A bind mount replaces the container's directory at that path with the host directory. Mounting an empty host folder over `/app` made `log_filter_cli.py` disappear, so `python` reported "can't open file". Keep code and output in different directories.

## Written answer

`python:3.13-slim` is preferred because it contains only the packages needed to run Python, so the image is much smaller (roughly 120-150 MB against about 1 GB for the full image, check with `docker images`), builds and downloads faster and has fewer packages that can contain vulnerabilities. The full image adds compilers and tools the script does not need.

## Checkpoints

- `docker images` size shown; `docker run` produces `out/output.csv` on the host; the file is identical to the host-run output.
- Without `-v`, the file is lost when the container stops (ask students to try).

**CCSP Lab 13:** same Dockerfile with `CMD [... "--level", "WARN"]`, image `cloud-log-filter:1.0`. **Healthcheck answer:** a run-once script has no long-lived process to probe, so a healthcheck only makes sense for a service. For a service add `HEALTHCHECK --interval=30s --timeout=3s CMD python -c "import urllib.request,sys; urllib.request.urlopen('http://localhost:8080/health', timeout=2)" || exit 1`, and expose a `/health` endpoint; for this batch job, the exit code is the health signal.

# Lab 14: Automating Container Deployment

**Recommended lecture week: Week 14 (second half) or Week 15**

> **Verification status: Reasoned**
> The commands are standard Docker Registry usage; the `out2` mount has the same defect as Lab 13.

```
docker run -d -p 5000:5000 --restart=always --name local-registry registry:2
docker tag forensic-log-filter:1.0 localhost:5000/forensic-log-filter:1.0
docker push localhost:5000/forensic-log-filter:1.0
curl http://localhost:5000/v2/_catalog
#   {"repositories":["forensic-log-filter"]}
curl http://localhost:5000/v2/forensic-log-filter/tags/list
#   {"name":"forensic-log-filter","tags":["1.0"]}
docker rmi forensic-log-filter:1.0 localhost:5000/forensic-log-filter:1.0
docker pull localhost:5000/forensic-log-filter:1.0
mkdir -p out2
docker run --rm --user "$(id -u):$(id -g)" -v "$PWD/out2:/app/out"
localhost:5000/forensic-log-filter:1.0
```

### Written note (model answer)

**Advantage:** the images stay inside the controlled network, there are no external pull-rate limits and no account is needed. **Limitation:** you must run, secure, back up and update the registry yourself (the lab registry has no TLS or authentication), and it offers no built-in vulnerability scanning or high availability.

### Checkpoints

- Catalog output lists the repository; the pull-then-run screenshot shows the image fetched from `localhost:5000`; output file appears in `out2/`.

> **Cloud track (CCSP-5213, BS Cloud Application Development & Operations)**
> **CCSP Lab 14: replace MinIO with Moto.** MinIO's community images are no longer published and its repository is in maintenance mode. Use the S3 emulator already used in the Cloud notes:

```
pip install "moto[server]" boto3
moto_server -p 5000 &
python3 - <<'PY'
import boto3
s3 = boto3.client("s3", endpoint_url="http://localhost:5000", region_name="us-east-1",
                  aws_access_key_id="test", aws_secret_access_key="test")
s3.create_bucket(Bucket="pipeline-artifacts")
s3.upload_file("out/output.csv", "pipeline-artifacts", "output.csv")
print([o["Key"] for o in s3.list_objects_v2(Bucket="pipeline-artifacts")["Contents"]])
s3.download_file("pipeline-artifacts", "output.csv", "downloaded.csv")   # the step
the manual never does
PY
cmp out/output.csv downloaded.csv && echo "identical"
```

Expected: `['output.csv']` then `identical`. Replace the "MinIO console screenshot" with the `list_objects_v2` output. If you still want MinIO, build it from source or use a maintained image such as Chainguard's, and note that the full web console is no longer part of the community edition. Reasoned.

**Comparison note (model answer):** a self-hosted S3 emulator is free and has no usage caps, but it has no durability, scaling, IAM depth or support; a managed free tier gives real durability and security controls but has limits and requires an account and billing controls.

# Lab 15: CI/CD Pipeline Automation

**Recommended lecture week:** Week 15

> **Verification status: Reasoned for Jenkins; YAML syntax validated**
> The manual's Jenkins container has no Docker CLI, so `docker build` fails. The solution builds a small Jenkins image, adds a Trivy gate and offers GitHub Actions as the route matching the lecture notes and Assignment 3.

## Route A: Jenkins (fixed)

`Dockerfile.jenkins`:

```
FROM jenkins/jenkins:lts-jdk21
USER root
COPY --from=docker:cli /usr/local/bin/docker /usr/local/bin/docker
USER jenkins
docker build -f Dockerfile.jenkins -t jenkins-docker:lts .
docker volume create jenkins_home
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --group-add "$(stat -c %g /var/run/docker.sock)" \
  -v "$HOME/forensic-scripting/lab13:/repo:ro" \
  jenkins-docker:lts
docker exec jenkins git config --global --add safe.directory /repo
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

In the job use **Pipeline script from SCM**, Git, repository URL `/repo` (or a GitHub/Gitea URL), script path `Jenkinsfile`. Commit the Jenkinsfile in `lab13` (not `lab15`).

`Jenkinsfile` with a polling trigger and a vulnerability gate (pin the Trivy version you tested):

```
pipeline {
  agent any
  triggers { pollSCM('H/2 * * * *') }
  environment {
    IMAGE = "localhost:5000/forensic-log-filter"
    TAG = "${env.BUILD_NUMBER}"
  }
  stages {
    stage('Build') { steps { sh 'docker build -t $IMAGE:$TAG .' } }
    stage('Test') {
      steps { sh 'docker run --rm $IMAGE:$TAG --input sample_data.csv --output
/tmp/out.csv --level ERROR' }
    }
    stage('Scan') {
      steps {
        sh 'docker run --rm -v /var/run/docker.sock:/var/run/docker.sock
aquasec/trivy:<PINNED_VERSION> image --exit-code 1 --severity HIGH,CRITICAL --ignore-
unfixed $IMAGE:$TAG'
      }
    }
    stage('Push') { steps { sh 'docker push $IMAGE:$TAG' } }
  }
  post {
    success { echo 'Pipeline completed: image built, tested, scanned and pushed.' }
    failure { echo 'Pipeline failed: check the console output.' }
  }
}
```

### Route B: GitHub Actions (matches the lecture notes)

```
name: CI
on:
  push:
  pull_request:
permissions:
  contents: read
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: python -m py_compile log_filter_cli.py
      - run: python log_filter_cli.py --input sample_data.csv --output /tmp/out.csv --
level ERROR
  build-and-scan:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t forensic-log-filter:${{ github.sha }} .
      - uses: aquasecurity/trivy-action@<PINNED_TAG_OR_SHA>
        with:
          image-ref: forensic-log-filter:${{ github.sha }}
          severity: CRITICAL,HIGH
          ignore-unfixed: true
          exit-code: "1"
```

The YAML above parses correctly (validated). Replace the placeholders with released versions you have checked, and check the current major versions of `actions/checkout` and `actions/setup-python`.

### Checkpoints

- A green run (Jenkins or GitHub) with Build, Test, Scan and Push stages; the Test stage log shows the container ran the script.
- `curl http://localhost:5000/v2/forensic-log-filter/tags/list` lists the new build tag.
- Student can explain why Test comes before Build/Push and what `exit-code: 1` does in the scan stage.

**CCSP Lab 15:** identical with image `cloud-log-filter`, `--level WARN` and job name `cloud-log-filter-pipeline`.

# Lab 16: Capstone Lab: Complete Forensic Pipeline

**Recommended lecture week:** Week 16

> **Verification status: Executed (all stages except faasd and Docker)**
> In the manual the stages do not connect: the alert script ignores the scan result, `raw.csv` and the Jenkinsfile path are missing, and `curl` and `127.0.0.1` fail inside the container. The corrected pipeline below ran end to end in the sandbox using the local harness for the IOC scan.

## Repository layout

```
capstone/
  data/raw.csv
  scripts/log_filter_cli.py     (Lab 3, corrected)
  scripts/alert_from_scan.py
  faas/handler.js  faas/harness.js   (Lab 8)
  docker/Dockerfile   jenkins/Jenkinsfile   .github/workflows/ci.yml
  run_pipeline.sh   diagram.png   README.md
```

## data/raw.csv

```
timestamp,host,level,message
2026-03-04T10:00:00,host-1,INFO,user admin logged in
2026-03-04T10:01:00,host-2,ERROR,outbound connection to 203.0.113.99 flagged
2026-03-04T10:02:00,host-3,ERROR,disk almost full
2026-03-04T10:03:00,host-2,WARN,process malicious.exe started
```

## scripts/alert_from_scan.py (reads the scan result)

```
import json, os, smtplib, sys
from email.mime.text import MIMEText

path = sys.argv[1]
with open(path, encoding="utf-8") as f:
    result = json.load(f)
result = result.get("body", result)            # accept the raw function body or the
harness envelope
if not result.get("matched"):
    print("No IOC matches, no alert sent")
    sys.exit(0)

msg = MIMEText(f"IOC hits: {result['hits']}")
msg["Subject"] = "Forensic Alert: IOC Match"
msg["From"], msg["To"] = "soc-automation@lab.local", "analyst@lab.local"
host, port = os.getenv("MAIL_HOST", "localhost"), int(os.getenv("MAIL_PORT", "1025"))
try:
    with smtplib.SMTP(host, port, timeout=10) as server:
        server.send_message(msg)
except (OSError, smtplib.SMTPException) as exc:
    sys.exit(f"Alert failed: {exc}")
print("Alert sent:", result["hits"])
```

## run_pipeline.sh

```
#!/usr/bin/env bash
# Capstone orchestrator: stops at the first failure and logs each stage with a UTC
timestamp.
set -euo pipefail
cd "$(dirname "$0")"
mkdir -p data out
log() { echo "$(date -u +%FT%TZ) $*" | tee -a out/pipeline.log; }

log "[1/4] Filtering raw logs"
```

```
python3 scripts/log_filter_cli.py --input data/raw.csv --output data/filtered.csv --
level ERROR

log "[2/4] Scanning for IOC matches"
if [ -n "${SCAN_URL:-}" ]; then                      # production: the faasd function,
e.g. http://host.docker.internal:8080/function/ioc-alert
  curl -fsS -X POST "$SCAN_URL" --data-binary @data/filtered.csv >
data/scan_result.json
else                                                 # no faasd available: run the
same handler through the local harness
  node faas/harness.js faas/handler.js "$(cat data/filtered.csv)" >
data/scan_result.json
fi

log "[3/4] Sending alert if matches found"
python3 scripts/alert_from_scan.py data/scan_result.json

log "[4/4] Archiving results"
ARCHIVE="out/archive_$(date -u +%Y%m%d_%H%M%S).tar.gz"
tar -czf "$ARCHIVE" data/
sha256sum "$ARCHIVE" >> out/SHA256SUMS
log "Pipeline complete: $ARCHIVE"
```

**Output observed when the corrected code was run:**

```
2026-10-09T10:53:38Z [1/4] Filtering raw logs
2 rows with level=ERROR written to data/filtered.csv (0 skipped)
2026-10-09T10:53:38Z [2/4] Scanning for IOC matches
2026-10-09T10:53:38Z [3/4] Sending alert if matches found
Alert sent: ['203.0.113.99']
2026-10-09T10:53:38Z [4/4] Archiving results
2026-10-09T10:53:38Z Pipeline complete: out/archive_20261009_105338.tar.gz
$ cat data/scan_result.json
{"status":200,"body":{"scanned":true,"hits":["203.0.113.99"],"matched":true}}
$ sha256sum -c out/SHA256SUMS
out/archive_20261009_105338.tar.gz: OK
```

## docker/Dockerfile for the pipeline (Reasoned)

```
FROM python:3.12-slim
RUN apt-get update && apt-get install -y --no-install-recommends curl \
 && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY . .
RUN useradd --create-home appuser && mkdir -p data out && chown -R appuser /app
USER appuser
ENTRYPOINT ["./run_pipeline.sh"]
docker build -f docker/Dockerfile -t forensic-pipeline:1.0 .
docker run --rm --add-host=host.docker.internal:host-gateway \
  -e SCAN_URL=http://host.docker.internal:8080/function/ioc-alert \
  -e MAIL_HOST=host.docker.internal \
  -v "$PWD/out:/app/out" forensic-pipeline:1.0
```

Inside a container `127.0.0.1` means the container, so the function and the mail server on the host are reached by `host.docker.internal`. `curl` is installed because the slim image does not include it.

## Written reflection (model answer)

The hardest part to integrate was the container boundary. Each tool worked alone, but inside the image `localhost` pointed at the wrong machine and a volume mounted over `/app` hid the code. I solved it by reading the error, replacing `127.0.0.1` with a configurable `SCAN_URL`, and mounting only the output folder. I also learned that data formats must agree between stages: the alert script had to read the scan result instead of a hard-coded line.

## Checkpoints

- Diagram shows raw logs, filter, IOC scan, alert, archive, CI/CD.
- A single run produces the log, the e-mail and a hashed archive; the CI run is green.
- Demo answers: "what happens if the scan stage fails?" (the script stops because of `set -e`).

# Part II: CCSP-5213 Solutions (Labs 1-16)

Labs 1-7 and 11-13, 15 follow the DFSP solutions above with the differences already noted in each lab. This part gives the Cloud-specific material: Lab 8 handler (given in Lab 8 above), Lab 9, 10, 14 variants (given in Labs 9, 10, 14 above) and the Cloud capstone.

| CCSP lab | Solution |
|---|---|
| 1-4 | Same as DFSP Labs 1-4 (cloud wording; folder `~/cloud-scripting`) |
| 5 | DFSP Lab 5 plus the emulator CLI step (`aws --endpoint-url ...`) |
| 6-7 | Same as DFSP Labs 6-7 |
| 8 | Usage-alert handler and harness outputs (see Lab 8, Cloud box) |
| 9 | Same workflow; emulator variant noted in Lab 9 |
| 10 | Five-resource consolidated cost alert (Lab 10, Cloud box) |
| 11 | Same, with the corrected retry (Lab 11, Cloud box) |
| 12-13 | Same as DFSP Labs 12-13 (healthcheck answer in Lab 13) |
| 14 | Moto S3 upload and download (Lab 14, Cloud box) |
| 15 | Same as DFSP Lab 15 |
| 16 | Cloud capstone below |

## Lab 16 (CCSP): Complete Cloud-Native Pipeline

> **Verification status: Executed (all stages except the S3 upload, Docker and Jenkins)**
> The manual posts a CSV to a function that calls `JSON.parse`, which throws (confirmed). The corrected pipeline converts each row to JSON, calls the function once per row and keeps going when a row is invalid.

### data/usage.csv

```
resource,costUsd,level
vm-01,72.5,WARN
db-02,20.0,INFO
vm-03,130.0,WARN
bad-row,abc,WARN
```

### scripts/check_budgets.py

```
"""Send each filtered usage row to the usage-alert function and collect the
answers."""
import csv, json, os, subprocess, sys, urllib.request

src, dst = sys.argv[1], sys.argv[2]
url = os.getenv("SCAN_URL")                        # e.g.
http://host.docker.internal:8080/function/usage-alert
results = []
with open(src, newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        payload = json.dumps({"resource": row["resource"], "costUsd": row["costUsd"]})
        if url:
            req = urllib.request.Request(url, data=payload.encode(),
headers={"Content-Type": "application/json"})
            with urllib.request.urlopen(req, timeout=10) as r:
                results.append(json.load(r))
```

```
        else:
            out = subprocess.run(["node", "faas/harness.js", "faas/handler.js",
payload], capture_output=True, text=True, check=True)
            results.append(json.loads(out.stdout)["body"])
with open(dst, "w", encoding="utf-8") as f:
    json.dump(results, f, indent=2)
errors = [r for r in results if "error" in r]
over = [r for r in results if r.get("overBudget")]
print(f"{len(results) - len(errors)} resources checked, {len(errors)} rejected as
invalid, {len(over)} over budget: {[r['resource'] for r in over]}")
```

**Output observed when the corrected code was run:**

```
$ python3 scripts/check_budgets.py data/usage.csv data/scan_result.json
3 resources checked, 1 rejected as invalid, 1 over budget: ['vm-03']
$ THRESHOLD_USD=50 python3 scripts/check_budgets.py data/usage.csv
data/scan_result.json
3 resources checked, 1 rejected as invalid, 2 over budget: ['vm-01', 'vm-03']
```

## scripts/alert_from_scan.py (Cloud)

```
import json, os, smtplib, sys
from email.mime.text import MIMEText

with open(sys.argv[1], encoding="utf-8") as f:
    results = json.load(f)
over = [r for r in results if r.get("overBudget")]
if not over:
    print("Within budget, no alert sent")
    sys.exit(0)

msg = MIMEText("Resources over budget:\n" + "\n".join(f"- {r['resource']}:
${r['costUsd']:.2f}" for r in over))
msg["Subject"] = f"Cloud Cost Alert: {len(over)} resource(s) over budget"
msg["From"], msg["To"] = "cost-automation@lab.local", "ops@lab.local"
try:
    with smtplib.SMTP(os.getenv("MAIL_HOST", "localhost"), int(os.getenv("MAIL_PORT",
"1025")), timeout=10) as s:
        s.send_message(msg)
except (OSError, smtplib.SMTPException) as exc:
    sys.exit(f"Alert failed: {exc}")
print("Alert sent for:", [r["resource"] for r in over])
```

## scripts/upload_to_s3.py (the upload script the manual never provides; Reasoned)

```
import os, sys, boto3
s3 = boto3.client("s3", endpoint_url=os.getenv("S3_URL", "http://localhost:5000"),
region_name="us-east-1",
                  aws_access_key_id=os.getenv("AWS_ACCESS_KEY_ID", "test"),
                  aws_secret_access_key=os.getenv("AWS_SECRET_ACCESS_KEY", "test"))
bucket, path = "pipeline-artifacts", sys.argv[1]
try:
    s3.create_bucket(Bucket=bucket)
except s3.exceptions.BucketAlreadyOwnedByYou:
    pass
s3.upload_file(path, bucket, os.path.basename(path))
print("uploaded", path, "to", bucket)
```

## run_pipeline.sh (Cloud)

```
#!/usr/bin/env bash
# Cloud capstone orchestrator: stops at the first failure; logs each stage with a UTC
timestamp.
set -euo pipefail
cd "$(dirname "$0")"
```

```
mkdir -p data out
log() { echo "$(date -u +%FT%TZ) $*" | tee -a out/pipeline.log; }

log "[1/5] Filtering usage data"
python3 scripts/log_filter_cli.py --input data/usage.csv --output data/filtered.csv --
level WARN
log "[2/5] Checking budgets"
python3 scripts/check_budgets.py data/filtered.csv data/scan_result.json
log "[3/5] Alerting if over budget"
python3 scripts/alert_from_scan.py data/scan_result.json
ARCHIVE="out/archive_$(date -u +%Y%m%d_%H%M%S).tar.gz"
log "[4/5] Archiving"
tar -czf "$ARCHIVE" data/
sha256sum "$ARCHIVE" >> out/SHA256SUMS
if [ -n "${S3_URL:-}" ]; then                          # skipped when no S3 endpoint is
configured
  log "[5/5] Uploading to the S3 endpoint"
  python3 scripts/upload_to_s3.py "$ARCHIVE"
fi
log "Pipeline complete: $ARCHIVE"
```

**Output observed when the corrected code was run:**

```
$ THRESHOLD_USD=50 ./run_pipeline.sh
2026-10-09T10:58:27Z [1/5] Filtering usage data
3 rows with level=WARN written to data/filtered.csv (0 skipped)
2026-10-09T10:58:27Z [2/5] Checking budgets
2 resources checked, 1 rejected as invalid, 2 over budget: ['vm-01', 'vm-03']
2026-10-09T10:58:27Z [3/5] Alerting if over budget
Alert sent for: ['vm-01', 'vm-03']
2026-10-09T10:58:28Z [4/5] Archiving
2026-10-09T10:58:28Z Pipeline complete: out/archive_20261009_105828.tar.gz
```

The pipeline ran end to end in the sandbox with the upload stage skipped (it only runs when `S3_URL` is set, for example `S3_URL=http://localhost:5000` with Moto running). The data flows between stages as JSON, an invalid row is rejected but does not stop the run, and the e-mail lists the resources over budget.

## Checkpoints

- No stage receives data in a format it cannot parse; an invalid row is reported but does not stop the run.
- Archive appears in the S3 bucket listing; its hash is in `SHA256SUMS`.
- Jenkins or GitHub Actions run is green, and the demo covers the failure case (what if the function is down?).

## 2. Marking Notes

- **Common student errors to watch for:** counting with `... is None` (Lab 3), moving the script itself (Lab 4), mounting over `/app` (Labs 13-14, 16), using `127.0.0.1` inside a container (Lab 16), catching the built-in `ConnectionError` (CCSP Lab 11), committing `.env` or `.venv`.
- **Evidence:** every screenshot should show the system clock; ask for one personalised value (threshold, `--top`, resource name prefix).
- **Reproducible data:** the seeded generator (`random.seed(42)`) gives every student 12 ERROR, 16 WARN and 22 INFO rows, so counts can be checked in seconds.
- **Time plan:** Labs 8, 13-16 need the most set-up time. Pre-pull `python:3.12-slim`, `registry:2` and the Jenkins image on the lab network.